First Assignment

Role: Product Engineer

Instructions Summary: Estimated completion time: 2 hours. All sections mandatory. Submission naming: {Last4DigitsOfPhoneNumber} - {YourName} - FirstAssignment - ProductEngineer.pdf. Contents confidential.

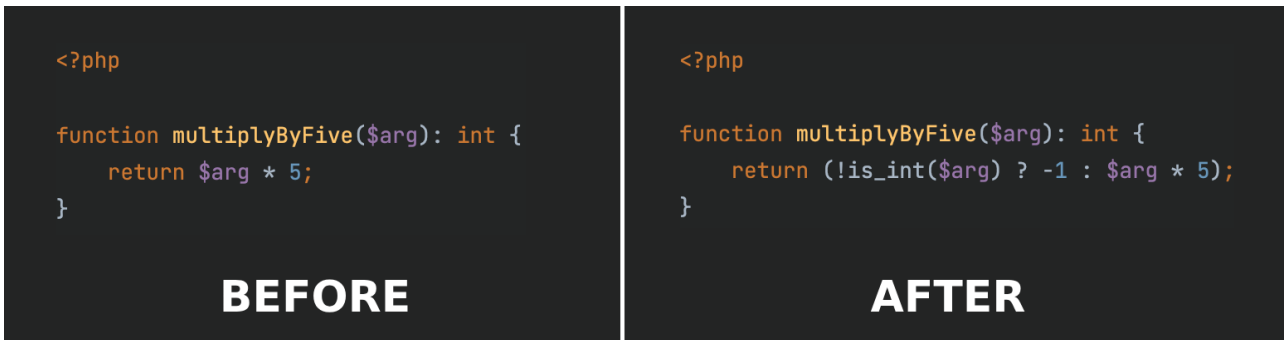
1. Candidate Details

Verified details matching application and public records:

Name:	Ashutosh Kumar Jaiswal
Phone:	+91 6207151423
Email:	ashutoshhjp00567@gmail.com
LinkedIn Profile URL:	https://www.linkedin.com/in/ashutosh-kumar-jaiswal/
GitHub URL:	https://github.com/Ashujais

2. Question: Code Change

Code Change Under Examination:



Recommended Git Commit Message:

```
fix(math): guard multiplyByFive against non-integer arguments

Add an explicit runtime check using is_int() to verify that $arg is an integer before
computing the product. If a non-integer value is provided, return -1 as an error indicator to
avoid unintended type coercion or runtime type errors under strict typing.
```

Bonus: Concerns Regarding the Final Version:

• **1. Sentinel Value Anti-Pattern (-1):** Returning -1 creates ambiguity between computational results and error states. If $arg = -0.2, mathematically -0.2 * 5 = -1. A caller checking if ($result === -1) cannot tell if an error occurred or if -1 was the genuine result. Moreover, in PHP, -1 evaluates to true in loose boolean checks (if (multiplyByFive('abc'))), allowing bad state to silently propagate.

• **2. Bypassing PHP's Native Type System:** The function specifies a return type hint (: int), but leaves the parameter $arg completely untyped, relying on manual runtime inspection (!is_int($arg)). In modern PHP (7.0+ / 8.x), the idiomatic and performant approach is declaring the parameter type directly: function multiplyByFive(int $arg): int with

declare(strict_types=1);. The Zend Engine enforces type contracts natively with zero manual overhead.

• **3. Violation of Fail-Fast Principle (Exceptions vs Error Codes):** Silently returning an arbitrary number forces every downstream consumer to write defensive checks. Modern software engineering best practice requires failing fast by throwing an InvalidArgumentException or TypeError when invalid data is supplied.

• **4. Numeric Float Inflexibility:** If a numeric float (e.g. 2.5) is passed, is_int(2.5) evaluates to false, returning -1 instead of 12.5. If decimal operations are valid, union types (int|float) should be supported.

• **5. Integer Overflow:** In PHP, if $arg * 5 exceeds PHP_INT_MAX (9,223,372,036,854,775,807 on 64-bit systems), PHP automatically promotes the result to a float. Because the return type hint is strictly : int, this triggers a fatal TypeError: Return value of multiplyByFive() must be of the type int, float returned.

What I Would Do Differently (Production-Grade Code):

```
<?php
declare(strict_types=1);

namespace App\Math;

use InvalidArgumentException;
use TypeError;

/**
* Multiplies an integer by five.
*
* @param int $arg The integer to multiply.
* @return int The product of $arg and 5.
* @throws TypeError If a non-integer is passed in strict typing mode.
*/
function multiplyByFive(int $arg): int {
return $arg * 5;
}

// Alternatively, if defensive handling of mixed untyped inputs is required:
function multiplyByFiveDefensive(mixed $arg): int {
if (!is_int($arg)) {
throw new InvalidArgumentException(
sprintf('Argument must be an integer, %s given.', get_debug_type($arg))
);
}
return $arg * 5;
}
```

3. Question: Task Approach

Scenario: You are asked to create a 'Newsletter Signup' page by your company's Sales team with no other inputs. What are your first steps to approach this work?

Strategic Overview: As a Product Engineer, building features without context often leads to wasted engineering cycles, poor conversion rates, or misaligned lead qualification. An underspecified request requires a structured, consultative approach that balances commercial sales objectives, UX friction, data privacy, and scalable technical architecture.

Step 1: Context Gathering & Commercial Alignment (Discovery)

• **Understand the Core Objective:** Clarify why Sales needs this page (e.g. B2B enterprise pipeline acceleration, webinar/event registrations, product launch updates, or general lead generation).

• **Target Audience & Channels:** Determine whether traffic comes from outbound cold outreach (SDRs), paid performance ads, organic blog traffic, or offline conference QR codes.

• **Define Success Metrics:** Align on target conversion rate (e.g., 3-5%), projected weekly subscriber volume, and qualification criteria for Marketing Qualified Leads (MQLs) vs. Sales Qualified Leads (SQLs).

Step 2: Define Requirements & Scope Boundaries (Functional & Non-Functional)

• **Form Field Strategy:** Balance lead qualification against conversion friction. Minimal MVP: Work Email. B2B Tier: First Name, Work Email, Company Name, and Job Role/Size.

• **Integrations & Lead Handoff:** Connect submissions directly to Sales CRM (HubSpot, Salesforce, Marketo), internal PostgreSQL backup table, and Slack alerts for enterprise domains.

• **Post-Signup Journey:** Immediate on-page confirmation UI, redirect to high-value lead magnet (e.g., Whitepaper/Report download), and automated welcome email with double opt-in.

• **Compliance & Privacy:** Implement strict GDPR, CAN-SPAM, and CCPA compliance (explicit opt-in consent checkbox, privacy policy link, 1-click unsubscribe mechanism, audit trail).

• **Security & Anti-Abuse:** Cloudflare Turnstile / reCAPTCHA v3 bot protection, honeypot fields, rate limiting per IP (max 5/min), and MX record validation to disallow disposable email domains.

Step 3: Rapid Prototyping & Wireframing

• Create a mobile-first, low-fidelity wireframe in Figma or interactive mockup showcasing headline, value proposition, form fields, social proof metrics, and confirmation states.

• Draft the technical API contract (POST /api/v1/newsletter/subscribe) and data schema.

Step 4: Stakeholder Alignment & Sign-Off

• Present the wireframe and proposed scope to the Sales lead in a 15-minute alignment sync.

• Establish agreement on MVP deliverables (launch within 5 days) vs. Phase 2 enhancements (A/B testing, personalized email drip flows).

Step 5: Technical Implementation (Iterative Engineering Sprint)

• **Frontend:** Semantic, accessible (WCAG 2.1 AA) responsive landing page built with modern framework (React/Next.js/Tailwind). Features instant client-side validation and responsive layout.

• **Backend API & Async Queue:** Express/FastAPI endpoint with rate limiting and payload validation. Asynchronous task queue (Redis/BullMQ) to sync CRM leads in background without blocking user response times (<200ms).

• **Analytics & Telemetry:** Instrument Segment/PostHog/GA4 event tracking (Pageview, Form Focus, Submit Attempt, Success, Error).

Step 6: QA, Cross-Browser Testing & Security Verification

• Test validation edge cases: duplicate submissions, malformed emails, uppercase strings, XSS/SQLi injection vectors, and slow 3G mobile networks.

• End-to-end integration test: Verify that submitted leads correctly populate in the Sales CRM with all attribution parameters (UTM source, medium, campaign) intact.

Step 7: Deployment & Staging Review

• Deploy to a staging environment for Sales team preview and acceptance sign-off.

• Production release via zero-downtime CI/CD deployment with Datadog/Sentry monitoring.

Step 8: Post-Launch Review & Iteration

• Monitor error rates, bounce rates, and funnel conversion metrics in real time.

• Conduct a 7-day post-launch review with the Sales team to assess lead volume and lead quality, followed by A/B copy tests on headlines and CTA buttons.

4. Question: Communication For Requirement Gathering

Write down your first communication (in email format) with the Sales team for their inputs, including timelines.

Subject: [Kickoff] Newsletter Signup Page – Requirements & Proposed 5-Day Launch Plan

Hi Sales Team,

Thanks for reaching out! A dedicated Newsletter Signup page is an excellent initiative to accelerate our inbound lead pipeline and engage prospective customers.

To ensure we build an experience that maximizes conversion while capturing the exact data your team needs for sales workflows, I've put together a recommended baseline along with a few key questions.

Rather than starting from a blank slate, here is our recommended **MVP (Phase 1) Proposal**: **Form Fields:** Work Email + First Name (keeps form friction low while enabling personalized SDR outreach). **CRM & Routing:** Real-time webhook sync into your CRM (HubSpot/Salesforce) tagged with *"Source: Newsletter"*, plus automated Slack notifications for high-value enterprise domain signups. **User Experience:** Instant on-page confirmation UI + automated transactional welcome email delivering our latest industry benchmark whitepaper. **Compliance & Security:** Bot protection via Cloudflare Turnstile, honeypot fields, and strict GDPR / CAN-SPAM compliance.
Key Inputs Needed From Your Side: Target Audience & Traffic Source: Who is the primary persona (e.g., Enterprise CTOs, Mid-market VP Sales, or general consumers), and where will visitors arrive from (cold outbound campaigns, paid LinkedIn ads, or social media)? **Required Lead Fields:** Are Work Email and First Name sufficient for MVP, or do you strictly require Company Name, Job Title, or Team Size upfront? *(Note: Each additional field typically reduces conversion rates by 5–10%.)* **CRM Properties & Lead Assignment:** Which CRM list or pipeline stage should these contacts be mapped to, and which sales rep / team should be assigned ownership? **Incentive / Copy:** Do you have specific headlines or a lead magnet ready (e.g., *"Get our 2026 Industry Report"*), or would you like engineering/marketing to provide placeholder copy for your review?
Proposed Timeline & Milestones (5-Day Delivery): Day 1 (Tomorrow, 2:00 PM): Consolidate feedback and lock in requirements. **Day 2:** Share interactive wireframe and API schema for your sign-off. **Day 3–4:** Engineering sprint (frontend landing page, backend API, CRM sync, and analytics tracking). **Day 5:** Staging walkthrough with the Sales team, QA verification, and Production Launch.
Could you share your thoughts by **tomorrow at 12:00 PM**? Alternatively, I am happy to hop on a quick 15-minute sync—feel free to grab time on my calendar [Calendar Link] or reply directly here.

Looking forward to launching this together!

Best regards,
Ashutosh Kumar Jaiswal
Product Engineer
ashutoshhjp00567@gmail.com | +91 6207151423

5. Exercise: UI/UX Assessment

Website: https://shareatdoorstep.com/donate/ | Task: Go through the donation flow and record observations categorized by Good, Bad, and Neutral with feature suggestions.

#	Summary	Description	Type
1	**Geolocation-First Funnel Gating**	The donation journey begins by detecting or asking for the user's location/pin code before asking for item details. This is an excellent usability pattern because it prevents users from going through a multi-step cataloging process only to discover at the end that their area is unserviceable.	**Good**
2	**Multi-Fulfillment Options (Doorstep / Drop / Campaigns)**	Offering three distinct pathways ('Pickup from Doorstep', 'Live Campaigns', and 'Drop yourself') caters effectively to diverse donor personas—from busy professionals willing to pay for convenience to budget-conscious donors who prefer free drop-offs.	**Good**
3	**Clear Item Condition & Age Tagging**	Allowing donors to specify item state ('Brand New', 'Barely Used', 'Well Used') and age brackets gives valuable operational data to sorting teams and partner NGOs, ensuring dignity for recipients and reducing logistics rejection.	**Good**
4	**Late Fee Disclosure / Price Shock at Checkout**	The platform communicates 'Donate' as a charitable act, but surprises the user with a mandatory convenience/pickup logistics fee only at the very final checkout step. This late pricing shock causes friction, drops trust, and triggers cart abandonment. Displaying transparent copy upfront (e.g., 'Doorstep pickup supported by a small Rs. 199 logistics contribution') would drastically reduce drop-offs.	**Bad**
5	**Mandatory Mobile OTP Barrier at Final Step**	Requiring users to enter their mobile number and wait for an SMS OTP right before placing the order introduces unnecessary drop-off friction. If SMS delivery is delayed or signal is weak, the entire donation flow fails. Implementing 1-click social login (Google OAuth) or delayed verification post-booking would improve checkout completion.	**Bad**
6	**Repetitive Modal Dialogs for Multi-Category Donors**	Adding multiple distinct items (e.g., clothes, books, and electronics) forces the user to navigate through repetitive modal overlays to input condition, brand, and age for each item individually. For a household decluttering session with 10+ items, this creates high cognitive fatigue.	**Bad**
7	**Opaque NGO / Cause Allocation During Booking**	The donor is not shown which specific NGO, orphanage, or community organization will receive their items during the booking flow. Showing paired NGO cards or impact stories during checkout would create an emotional bond and boost repeat donations.	**Neutral**
8	**Feature Suggestion: AI-Powered Photo Donation Estimation**	Instead of manually browsing categories and item dropdowns, allow donors to snap a picture of their donation box/bag. Computer vision can automatically estimate volume, category breakdown, and assign the appropriate vehicle/bag size for pickup.	**Neutral (Feature)**
9	**Feature Suggestion: Instant Environmental & Social Impact Counter**	Integrate dynamic impact metrics during item selection (e.g., 'Donating 10 clothes saves ~7,000 liters of water and provides warmth to 2 school children'). Gamifying and quantifying social/environmental impact in real time significantly increases motivation to finish the flow.	**Neutral (Feature)**

6. Exercise: Backend Design

Website: https://milaap.org/crowdfunding/fundraisers | Task: Guess the database schema of this page and fundraisers' listings.

1. Architectural Context & Modeling Strategy:
• **Platform:** Milaap (Online crowdfunding platform primarily for medical emergencies, personal causes, education, and memorials).
• **Database Choice:** Relational DBMS (PostgreSQL) is strictly recommended due to ACID transactional consistency required for monetary donations, strict audit trails, multi-currency processing, and tax exemption (80G) reporting.
• **Data Modeling Approach:** Highly normalized 3NF schema for transactional integrity, paired with strategic denormalization (cached aggregate counters raised_amount and donors_count on the campaigns table) to support sub-10ms query times on the high-traffic /crowdfunding/fundraisers listing page.

2. Entity-Relationship Overview:
[users] 1 ■■■■ N [campaigns] (as campaigner)
[categories] 1 ■■■■ N [campaigns]
[campaigns] 1 ■■■■ 1 [beneficiaries]
[campaigns] 1 ■■■■ N [campaign_media]
[campaigns] 1 ■■■■ N [donations]
[users] 1 ■■■■ N [donations] (as donor, optional/guest)
[campaigns] 1 ■■■■ N [campaign_updates]
[campaigns] N ■■■■ M [tags] (via campaign_tags)

3. Production Relational Schema (PostgreSQL DDL):

```
-- 1. USERS TABLE (Campaigners, Donors, Administrators)
CREATE TABLE users (
id BIGSERIAL PRIMARY KEY,
uuid UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
full_name VARCHAR(150) NOT NULL,
email VARCHAR(255) UNIQUE NOT NULL,
phone_number VARCHAR(20) UNIQUE,
password_hash VARCHAR(255),
profile_image_url TEXT,
kyc_status VARCHAR(30) DEFAULT 'PENDING' CHECK (kyc_status IN ('PENDING', 'VERIFIED', 'REJECTED')),
role VARCHAR(30) DEFAULT 'USER' CHECK (role IN ('USER', 'CAMPAIGNER', 'DONOR', 'ADMIN',
'VERIFIER')),
is_active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- 2. CATEGORIES TABLE (Medical, Memorials, Education, Emergency, etc.)
CREATE TABLE categories (
id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL UNIQUE,
slug VARCHAR(120) NOT NULL UNIQUE,
tag_line TEXT,
icon_url TEXT,
display_order INT DEFAULT 0,
is_active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 3. BENEFICIARIES TABLE (The patient or recipient of funds)
CREATE TABLE beneficiaries (
id BIGSERIAL PRIMARY KEY,
full_name VARCHAR(150) NOT NULL,
relationship_to_campaigner VARCHAR(50) NOT NULL, -- 'myself', 'family_member', 'friend',
'organisation'
age INT,
gender VARCHAR(20),
medical_condition TEXT,
hospital_name VARCHAR(200),
treating_doctor VARCHAR(150),
hospital_city VARCHAR(100),
verification_documents_url TEXT[],
is_verified BOOLEAN DEFAULT FALSE,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 4. CAMPAIGNS TABLE (Fundraisers - Powers the listing page cards)
CREATE TABLE campaigns (
id BIGSERIAL PRIMARY KEY,
uuid UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
slug VARCHAR(255) UNIQUE NOT NULL,
title VARCHAR(255) NOT NULL,
story_summary TEXT NOT NULL,
story_html TEXT,
campaigner_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
beneficiary_id BIGINT REFERENCES beneficiaries(id) ON DELETE SET NULL,
category_id INT NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
currency CHAR(3) DEFAULT 'INR' NOT NULL,
goal_amount NUMERIC(14, 2) NOT NULL CHECK (goal_amount > 0),
raised_amount NUMERIC(14, 2) DEFAULT 0.00 NOT NULL CHECK (raised_amount >= 0),
donors_count INT DEFAULT 0 NOT NULL CHECK (donors_count >= 0),
shares_count INT DEFAULT 0 NOT NULL CHECK (shares_count >= 0),
cover_image_url TEXT NOT NULL,
video_url TEXT,
tax_exemption_eligible BOOLEAN DEFAULT FALSE,
is_emergency BOOLEAN DEFAULT FALSE,
is_verified BOOLEAN DEFAULT FALSE,
featured_rank INT DEFAULT 0,
status VARCHAR(30) DEFAULT 'ACTIVE' CHECK (status IN ('DRAFT', 'UNDER_REVIEW', 'ACTIVE', 'PAUSED',
'COMPLETED', 'CLOSED', 'REJECTED')),
start_date TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
end_date TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 5. CAMPAIGN MEDIA TABLE (Gallery, hospital receipts, videos)
CREATE TABLE campaign_media (
id BIGSERIAL PRIMARY KEY,
campaign_id BIGINT NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
media_type VARCHAR(20) NOT NULL CHECK (media_type IN ('IMAGE', 'VIDEO', 'DOCUMENT')),
media_url TEXT NOT NULL,
caption VARCHAR(255),
display_order INT DEFAULT 0,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- 6. DONATIONS TABLE (Financial transaction ledger)
CREATE TABLE donations (
id BIGSERIAL PRIMARY KEY,
transaction_uuid UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
campaign_id BIGINT NOT NULL REFERENCES campaigns(id) ON DELETE RESTRICT,
donor_id BIGINT REFERENCES users(id) ON DELETE SET NULL,
donor_name VARCHAR(150) NOT NULL DEFAULT 'Anonymous',
donor_email VARCHAR(255),
donor_phone VARCHAR(20),
is_anonymous BOOLEAN DEFAULT FALSE,
currency CHAR(3) DEFAULT 'INR' NOT NULL,
amount NUMERIC(12, 2) NOT NULL CHECK (amount > 0),
tip_to_milaap NUMERIC(10, 2) DEFAULT 0.00 CHECK (tip_to_milaap >= 0),
gateway_fee NUMERIC(10, 2) DEFAULT 0.00,
net_campaign_amount NUMERIC(12, 2) GENERATED ALWAYS AS (amount) STORED,
payment_gateway VARCHAR(50) NOT NULL,
gateway_transaction_id VARCHAR(150) UNIQUE,
payment_status VARCHAR(30) DEFAULT 'INITIATED' CHECK (payment_status IN ('INITIATED', 'SUCCESS',
'FAILED', 'REFUNDED')),
wants_80g_receipt BOOLEAN DEFAULT FALSE,
pan_number_encrypted TEXT,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 7. CAMPAIGN UPDATES TABLE (Hospital progress, updates to backers)
CREATE TABLE campaign_updates (
id BIGSERIAL PRIMARY KEY,
campaign_id BIGINT NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
author_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
title VARCHAR(200) NOT NULL,
content_html TEXT NOT NULL,
image_url TEXT,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- 8. TAGS & CAMPAIGN_TAGS (Tax Benefit, Urgent, Milaap Assured)
CREATE TABLE tags (
id SERIAL PRIMARY KEY,
name VARCHAR(50) UNIQUE NOT NULL,
slug VARCHAR(60) UNIQUE NOT NULL,
badge_color VARCHAR(20)
);

CREATE TABLE campaign_tags (
campaign_id BIGINT NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
tag_id INT NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
PRIMARY KEY (campaign_id, tag_id)
);
```

4. Strategic Indexing & Query Optimization for /crowdfunding/fundraisers:

• **Composite Listing Index:** CREATE INDEX idx_campaigns_status_created ON campaigns (status, featured_rank DESC, created_at DESC) WHERE status = 'ACTIVE';

• **Category Filter Index:** CREATE INDEX idx_campaigns_category_status ON campaigns (category_id, status) WHERE status = 'ACTIVE';

• **Fast Slug Lookup:** CREATE UNIQUE INDEX idx_campaigns_slug ON campaigns (slug);
• **Full-Text Search Index:** CREATE INDEX idx_campaigns_fts ON campaigns USING GIN (to_tsvector('english', title || ' ' || story_summary));
• **Fast Donation Lookup:** CREATE INDEX idx_donations_campaign_status ON donations (campaign_id, payment_status) WHERE payment_status = 'SUCCESS';

5. Concurrency & Data Consistency Strategy:
• **Atomic Aggregate Increments:** The listing page displays cards with *'Rs. raised of Rs. goal'* and *'N supporters'*. Running SELECT SUM(amount), COUNT(*) FROM donations WHERE campaign_id = X across dozens of cards on every request would degrade database performance. Instead, successful payment gateway webhooks trigger atomic increments:
UPDATE campaigns SET raised_amount = raised_amount + NEW.amount, donors_count = donors_count + 1, updated_at = CURRENT_TIMESTAMP WHERE id = NEW.campaign_id;
• **Edge Caching Layer:** High-throughput listing endpoints (/crowdfunding/fundraisers) are cached in Redis with a 30-60 second TTL, combined with Cache-Control: public, s-maxage=60, stale-while-revalidate=30 headers.

That’s all, folks!